CS-523 SecretStroll Report

Bianca Rechsteiner, Tapdig Maharramli, Berrada Ismael

***Abstract*—This report presents SecretStroll, a privacy-preserving location-based service enabling anonymous user queries through attribute-based credentials (ABCs). Our implementation of anonymous authentication ensures the unlinkability between users and their requests. Through a fine-grained performance evaluation, we show that both computational and communication costs scale linearly with attribute count. We further examine privacy risks related to metadata leakage and demonstrate that dummy query injection significantly improves spatial anonymity. Finally, we show that even when using Tor, analyzing encrypted traffic metadata through network fingerprinting can still compromise user privacy, achieving over 90% accuracy in inferring user locations - underscoring the limitations of IP anonymization alone.**

I. Introduction

In this project, we were tasked to implement and evaluate an application, where a user can obtain nearby points of interest (POIs) in a privacy preserving way. That is, the service provider should not be able to link a request to a specific user.

Part 1 of the project was to do the implementation of the system using the Pointcheval-Sanders ABC scheme that we saw in the lecture. For this part, we follow the suggested steps in the handout.

In Part 2 we had to evaluate the privacy risks resulting from the source IP address included in the user queries. We run a privacy evaluation on simulated data and implemented dummy query injection as a mitigation strategy to reduce linkability based on source IP address metadata.

Finally, in Part 3 we considered the scheme with randomized user IP addresses. The task was to perform a network traffic analysis to deanonymize users based on the network traffic metadata. We collected Tor traffic traces for different locations and used a machine learning classifier to perform fingerprinting based solely on encrypted traffic metadata.

II. Attribute-based credential

First, let us talk about how we modeled the attributes that go into the credentials. An attribute has a label and a value. At the initialization of the issuer, the labels and also the order of the attributes are determined. This means that all credentials will contain all possible attributes. This has the advantage that users cannot be deanonymized based on the number of attributes or labels of attributes, since these are the same for all users. The value of an attribute defines the value that the attribute has. For example, in the `username` attribute the value of the username is stored, or in the attribute `restaurant` it is stored whether the user is subscribed to do queries for restaurants or not. To be able to use these attributes in the PS scheme, we need to convert them into a number $a_i \in \mathbb{Z}_p$, where p is the order of the groups. We concatenated the label and the value to one byte string, and obtained the a_i by converting the byte string to an integer modulo p.

In our implementation, we do not explicitly check that a user is actually eligible to obtain a credential for the subscriptions they request. That is, we did not implement a mechanism that makes sure that a user has paid for the subscriptions they request. We assume that if a user requests a subscription, they are authorized to get it. In a real-world implementation this is of course not enough, but since the payment mechanism is not really related to ABCs, we found this to be enough for this assignment.

Next, we want to talk about what attributes we choose for our secretstroll implementation.

- `secret key`: The secret key of the user is a random string of 16 characters chosen from capital letters and digits. It is chosen at the initialization of a user. This attribute makes sure that the credential is bound to a user. The attribute is part of the user attributes, since the user should never reveal its secret key.
- `username`: A user can choose its username itself in the registration process. The issuer then checks if the requested username and the username in the issuance request match. If this is the case, the issuer sets the username attribute to the requested value. The attribute is part of the issuer attributes. This is because in this way the issuer can make sure that the user does not claim to be someone else.
- `subscriptions`: A subscription attribute has the label of the name of the subscription, for example `restaurant`. The value of the attribute is `subscribed` or `not subscribed`, depending on whether the user wants to get a subscription for the corresponding subscription or not. The subscription attributes are also chosen by the user in the registration process, and the issuer checks if the subscriptions match the subscriptions in the issuance request. The subscription attributes are also issuer attributes. This enables the issuer to verify if a user is eligible to actually get the corresponding subscriptions. As mentioned above, we did not implement any such payment checks in our code.

The implementation of the ABC protocol required two non-interactive zero knowledge proofs. The first one is in the issuance protocol which was specified as

$$\pi_0 = PK\{(t, (a_i)_{i \in \mathcal{U}} : C = g^t \prod_{i \in \mathcal{U}} Y_i^{a_i}\}$$

. This proof makes sure that the commitment C to the user attributes $a_i \in \mathcal{U}$ was computed correctly. The second proof

was in the showing protocol and specified as

$$\pi_1 = PK\{(t, a_i)_{i\in\mathcal{H}} : \frac{e(\sigma_2', \tilde{g}) \cdot \prod_{i\in\mathcal{D}} e(\sigma_1', \tilde{Y}_i)^{-a_i}}{e(\sigma_1', \tilde{X})} = e(\sigma_1', \tilde{g})^t \prod_{i\in\mathcal{H}} e(\sigma_1', \tilde{Y}_i)^{a_i}\}$$

This proof is to verify that the randomized signature $\sigma' = (\sigma_1', \sigma_2')$ computed by the user is a valid signature. To generate those non-interactive zero-knowledge proofs, we first need a sigma protocol to which we can apply the Fiat-Shamir transform. We noted that the relations we want to proof with π_0 and π_1 are very similar to Pedersen commitments. Therefore, we adapted the sigma protocol from problem 2 in the exercise sheet about anonymous authentication [1], which was used to proof knowledge of a Pedersen commitment. The adapted sigma protocol is specified in figure 1.

Common input: $(\mathbb{G}, g, p), \mathcal{U}, \{Y_i\}_{i\in\mathcal{U}}, com$

Prover		**Verifier**
Input: $w = (t, \{a_i\}_{i\in\mathcal{U}})$		
$r_t \in_\$ \mathbb{Z}_p$		
$r_i \in_\$ \mathbb{Z}_p \quad \forall i \in \mathcal{U}$		
$R := g^{r_i} \prod_{i\in\mathcal{U}} Y_i^{r_i}$	$\xrightarrow{R}$	
		$c \in_\$ \mathbb{Z}_p$
	$\xleftarrow{c}$	
$s_t = r_t - c \cdot t$		
$s_i = r_i - c \cdot a_i$		
	$\xrightarrow{s_t, \{s_i\}_{i\in\mathcal{U}}}$	
		Verify:
		$R = com^c g^{s_t} \prod_{i\in\mathcal{U}} Y_i^{s_i}$

Fig. 1: Sigma Protocol for Fiat-Shamir Transform

Mapping the values for π_0 to the sigma protocol in Figure 1 is straightforward. The only variables that are named differently are the group G_1, the generator g_1, and the commitment C they correspond to $\mathbb{G}$, g, and com. For the proof π_1 we do the following mapping, on the left hand side we put the variable names from the sigma protocol and on the right hand side the corresponding values from the specification of π_1

$$\begin{aligned}
\mathbb{G} &:= G_T \\
g &:= e(\sigma_1', \tilde{g}) \\
p &:= p \\
\mathcal{U} &:= \mathcal{H} \\
Y_i &:= e(\sigma_1', \tilde{Y}_i) \\
com &:= \frac{e(\sigma_2', \tilde{g}) \cdot \prod_{i\in\mathcal{D}} e(\sigma_1', \tilde{Y}_i)^{-a_i}}{e(\sigma_1', \tilde{X})} \\
t &:= t
\end{aligned}$$

Now, that we specified how we can map π_0 and π_1 to the sigma protocol from Figure 1, we are ready to apply the Fiat-Shamir transform to the sigma protocol. When we do a Fiat-Shamir transform, we generate the challenge c from a hash of all public values defined up to the point we need to know c. In addition, we include a message m in c. In our case we generate

$$c := H(g||\{Y_i\}_{i\in\mathcal{U}}||com||R||m)$$

To generate the proof q for the relations, the prover computes all remaining values like in the interactive sigma protocol. q then consists of the challenge c and the randomizations of t and the a_i's, i.e. s_t and $\{s_i\}_{i\in\mathcal{U}}$.

$$q := (c, s_t, \{s_i\}_{i\in\mathcal{U}})$$

To verify such q the verifier computes

$$R_v = com^c g^{s_t} \prod_{i\in\mathcal{U}} Y_i^{s_i}$$

where the values of the exponents are the ones specified in q. Then they recompute the challenge

$$c_v = H(g||\{Y_i\}_{i\in\mathcal{U}}||com||R_v)$$

and check if $c_v = c$. If this is the case, they accept the proof, otherwise they don't.

The message m that is included in the hash for c is optional. It is used in proof π_1 to bind the proof to a specific message, which specifies the location the user wants to query. This ensures that a disclosure proof cannot be replayed for a location different from that intended by the user.

A. Test

Our comprehensive test suite validates both our implementation of the ABC scheme and its integration with the SecretStroll system. We tested the system incrementally as recommended in the handout, with tests written for each implementation phase. Our test suite, encompassing 15 tests, is organised into five main test classes, namely TestPSSignature, TestCredentialSystem, TestSecretStrollIntegration, TestFailureConditions, and TestCompleteCredentialLifecycle, covering different aspects of the system. We first test the core PS signature functionality by checking proper key generation, signature creation, and verification, including negative cases with tampered messages.

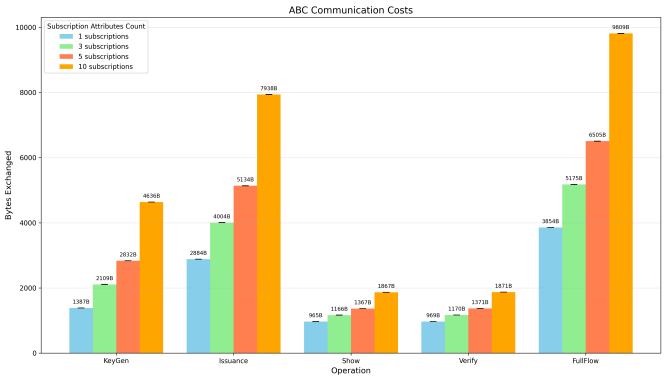

Fig. 2: Communication Costs of ABC Scheme (in bytes)

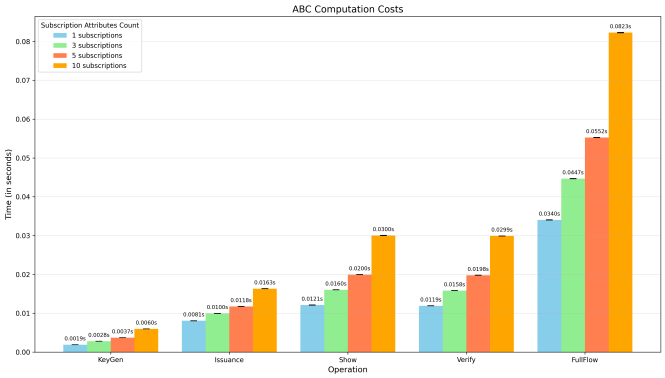

Fig. 3: Computation Costs of ABC Scheme (in seconds)

Then, we test the ABC scheme by verifying issuance request structure and ZK proof validity, followed by testing the issuer's blind signature generation and credential obtainment step. We also validate non-interactive zero-knowledge proof generation and verification.

After this, we test the integration with SecretStroll to ensure correct key generation, client's registration preparation with attribute handling, server's processing of the registration and blind signature generation, followed by end-to-end location query request workflow with selective disclosure.

In terms of failure conditions, we test for five scenarios. These scenarios include properly rejecting signing attempts with invalid credential parameters, invalid signature verification, detecting tampered ZK proof challenges, attempt of disclosure with non-existent attributes, and message binding verification, meaning that a tampered message will not pass disclosure proof verification which was created for another message. In the end, we test the complete lifecycle of the credential from key generation to its usage through selective disclosure.

B. Evaluation

We conducted a fine-grained performance evaluation of communication and computation costs for the following operations: key generation, issuance, showing the credential, and verification. We include both the mean and the standard error of the mean. The reported results are measured over 100 trials for 1, 3, 5, 10 number of attributes (corresponding to subscription types). We ran the performance evaluation inside the course-provided virtual machine. Apart from evaluating the performance of individual steps, we also include FullFlow, which is the entire execution of the ABC protocol from a user's perspective - from key generation to credential issuance to usage via ZK proof generation and verification in the showing and verification steps.

TABLE I: ABC Performance Summary (100 trials)

Operation	Attr.	Mean Time (s) ± SEM	Mean Bytes ± SEM
KeyGen	1	0.00192 ± 0.00001	1387.32 ± 0.37709
	3	0.00281 ± 0.00001	2109.19 ± 0.60897
	5	0.00371 ± 0.00001	2832.25 ± 0.64532
	10	0.00597 ± 0.00001	4636.94 ± 0.58635
Issuance	1	0.00808 ± 0.00002	2884.17 ± 1.14009
	3	0.00997 ± 0.00003	4004.37 ± 1.64333
	5	0.01176 ± 0.00002	5134.56 ± 2.00974
	10	0.01634 ± 0.00002	7938.03 ± 1.78605
Show	1	0.01212 ± 0.00003	965.91 ± 0.34026
	3	0.01605 ± 0.00002	1166.84 ± 0.57625
	5	0.01997 ± 0.00002	1367.09 ± 0.68403
	10	0.03005 ± 0.00004	1867.81 ± 0.57800
Verify	1	0.01193 ± 0.00002	969.91 ± 0.34026
	3	0.01585 ± 0.00002	1170.84 ± 0.57625
	5	0.01980 ± 0.00002	1371.09 ± 0.68403
	10	0.02992 ± 0.00002	1871.81 ± 0.57800
FullFlow	1	0.03405 ± 0.00005	3854.08 ± 1.18199
	3	0.04467 ± 0.00004	5175.21 ± 1.81583
	5	0.05524 ± 0.00004	6505.65 ± 2.19363
	10	0.08228 ± 0.00006	9809.84 ± 1.96498

Figure 2 and Figure 3 represent communication costs (in bytes) and computation costs (time in seconds) respectively. We also include Table I, demonstrating the mean and standard error of the mean values over 100 measurements for each attribute count of 1, 3, 5, and 10. Note that when we say attribute count, we mean the number of subscription types that are disclosed in the location query, assuming that they have been subscribed to.

The key takeaways from our evaluation demonstrate that both communication and computation costs scale approximately linearly with the number of attributes in our design. The standard errors of the mean values remain consistently small across all measurements, indicating stable and predictable performance characteristics of our ABC scheme implementation.

III. (De)Anonymization of User Trajectories

A. Privacy Evaluation

We assume that the adversary is the service provider, who observes the user's IP address, the timestamp, geographic coordinates, and the requested POI type for every query. The adversary is passive but strategic, aggregating metadata over time to infer user behavior.

Our evaluation focuses on three attacks: (1) inferring users' home locations from repeated nighttime queries with similar coordinates, (2) detecting likely work locations based on

repeated queries to food-related POIs during 9am–5pm, and (3) profiling interests by counting frequent POI types per IP.

Visual analysis confirmed the feasibility of these attacks. Temporal distributions revealed strong daily patterns and POI histograms exposed repeated user preferences. Using k-means clustering, we localized dense activity areas per user. A spatial k-anonymity analysis showed that 4.4% of real queries were isolated ($k < 5$ within 500m), indicating significant privacy leakage.

B. Defences

To mitigate location traceability, we implemented a client-side defence based on dummy query injection. For every real query with $k < 5$, we injected additional queries with similar metadata (IP, POI type, timestamp), perturbed by Gaussian noise within a 0.5 km radius. The goal was to raise spatial anonymity to $k = 8$.

After recomputing anonymity scores, only 0.9% of real queries remained under $k = 5$. This shows a significant privacy gain with minimal overhead. Dummy queries were not sent to the server, preserving utility in terms of service accuracy. The main cost lies in client-side computation and potential bandwidth usage for future extensions.

This defence addresses spatial isolation but not interest profiling or timing-based inference. It represents a lightweight and tunable mechanism that improves user privacy without degrading functionality. Future work could explore mix zones or shared dummy strategies to further enhance protection.

IV. Cell Fingerprinting via Network Traffic Analysis

A. Implementation details

To conduct this fingerprinting attack, we developed and used a script named `data_generation.py`, which automates traffic collection through the Tor network. For each cell ID (1 to 100), we issued 100 queries, collecting a total of 10,000 traffic traces in the form of `.pcap` files. These captures were done using `scapy` in a separate thread, with precise timing controls to ensure only the relevant packets were recorded. Data collection took around 42 hours, resulting in around 4.4 GB `.pcap` files in total.

From each trace, we extracted statistical features representing incoming and outgoing flows: number of packets, packet sizes (mean, sum, min, max, std), inter-arrival times (max/std), flow durations, and TCP flags (FIN, SYN, PSH, ACK) per direction. All features were computed on the packet metadata only, excluding payloads.

We trained a Random Forest classifier on the extracted features and evaluated it using a stratified 10-fold cross-validation.

B. Evaluation

The classifier achieved the following metrics after 10-fold cross-validation:

- Accuracy: **92.95%**
- Precision (macro): **93.62%**
- Precision (micro): **92.95%**
- F1-score (macro): **92.66%**
- F1-score (micro): **92.95%**

These values confirm the attack's effectiveness in correctly identifying the queried cell solely from metadata patterns.

C. Feature Mapping

The input vector for each trace includes 26 features, extracted from packet metadata only (no payload inspection). These features cover both incoming and outgoing directions and include: number of packets, total size, mean/min/max/std of packet sizes, maximum and standard deviation of inter-arrival times, flow durations, and TCP flags (FIN, SYN, PSH, ACK). All features are computed separately for outgoing and incoming traffic.

D. Feature Importance Analysis

We analyzed the internal feature importance scores from the Random Forest model to determine which inputs contribute the most to the classification. The top features include:

- Standard deviation of incoming packet sizes
- Total size of incoming flows
- Timing variations (e.g., `in_std_delta`)

Detailed importance values are shown in Table II. These results indicate that variations in flow size and timing are key indicators for identifying cells.

TABLE II: Top-ranked features by importance in the Random Forest classifier.

Feature	Importance (%)
`in_total_size`	16.7%
`in_std_size`	14.9%
`in_std_delta`	13.6%
`out_max_size`	11.3%
`in_max_delta`	9.8%
`incoming_packets`	8.7%

E. Discussion and Countermeasures

Our classifier performed strongly, achieving nearly 93% accuracy across 100 classes. The close match between macro and micro scores shows robustness and a balanced classification across all grid cells. The most important features, according to the Random Forest's internal ranking, include the standard deviation of incoming packet sizes, the total size of incoming flows, and timing variations—confirming that network traffic patterns inherently leak location-related information.

The main factor influencing performance is the determinism in response size and structure: each POI list sent by the server contributes to a unique traffic signature. While Tor encrypts and reroutes packets, it does not hide observable features such as flow length, packet count, or timing.

To mitigate such attacks, we recommend the following countermeasures:

- **Fixed-size responses:** pad responses so that all queries produce the same observable traffic length.

- **Dummy traffic injection:** generate noise packets that alter the traffic shape.
- **Timing obfuscation:** delay or reorder packets to hide timing correlations.

While these defences add overhead, they are necessary to protect against metadata-based deanonymization attacks. Our results show that Tor alone is insufficient for full location privacy if higher-layer protocols leak traffic patterns.

Contributions

- **Bianca**: Part 1: implementation of credentials.py and stroll.py, writing the corresponding parts in the report.
- **Ismael**: implemented and documented Parts 2 and 3 of the project. This included writing the script for data collection `data_generation.py`, and implementation of `fingerprint.py` and `part2.ipynb`, performing initial runs to validate the approach for part 3, and obtaining preliminary results for the fingerprinting classifier. Also wrote the corresponding sections in the report, summarizing the methodology, evaluation metrics, and key insights.
- **Tapdig**: Developed comprehensive test suites, conducted performance evaluation of the ABC scheme, and performed final data collection for network traffic analysis in Part 3.

References

[1] T. Stadler and C. Troncoso, "Anonymous authentication exercises," 2025, exercise sheet for CS-523: Advanced Topics on Privacy-Enhancing Technologies. [Online]. Available: https://moodle.epfl.ch/pluginfile.php/3320997/mod_folder/content/0/Privacy-preserving%20Authentication%20Theory%20Exercise%20Solutions.pdf